НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»
МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ІНСТИТУТ КОМП'ЮТЕРНИХ СИСТЕМ
КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ

Протокол лабораторної роботи №1

«Розгортання середовища та порівняльний аналіз мовних моделей»

з дисципліни: «Обчислювальні методи»

Виконав: Студент групи АІ-242
Вологжанін Д.Р.

Перевірив: ас. Баранов О.М.

Одеса – 2026

Завдання 1. Ціна роздумів: міркувальна vs інструктивна модель

Таблиця 1: Фактичне питання («Столиця України?»)

Показник	**qwen3:4b (міркувальна)**	**llama3.2:3b (інструктивна)**
response	Kyiv (Київ)	Київ
eval_count	2169	5
Час, с	111.98	6.85
Токенів/с	19.4	0.7

Токенів/с розраховано як eval_count / Час, с [2]. Для llama3.2:3b у цьому запиті значення (0.7) неінформативне: згенеровано лише 5 токенів, і час відповіді визначається не швидкістю генерації.

Таблиця 2: Задача на міркування (Ручка і зошит)

Показник	**qwen3:4b (міркувальна)**	**llama3.2:3b (інструктивна)**
Правильних із 5	5	0 (відповідала неправильно)
eval_count	1274	170
Час, с	59.63	4.98
Токенів/с	21.4	34.1

Токенів/с розраховано як eval_count / Час, с [2].

Відповіді чат-бота

1. Куди пішли ці токени?

Понад дві тисячі «зайвих» токенів у qwen3:4b були витрачені на прихований блок роздумів (thinking) [7]. Модель спочатку розмірковує «про себе», перевіряючи власні припущення, і тільки після цього формує коротку фінальну відповідь для користувача.

2. На яких типах задач міркувальні моделі дають перевагу над інструктивними, а на яких їх використання невиправдане?

Найбільша перевага проявляється в математичних і логічних задачах та під час написання коду, де потрібен покроковий розбір (саме тому Qwen розв’язала 5 із 5 задач) [4]. Натомість для простих фактичних запитань, як-от «столиця країни», вони недоцільні: час і обчислювальні ресурси витрачаються марно.

3. Чим відрізняється процес навчання міркувальної моделі від інструктивної?

Інструктивну модель навчають одразу видавати найімовірнішу правильну відповідь на запит. Міркувальну ж додатково донавчають на даних із покроковими розв’язаннями (підхід Chain of Thought) [4], через що вона звикає вибудовувати логічний ланцюжок перед остаточним висновком.

4. Чи означає більша кількість згенерованих токенів у міркувальної моделі, що вона «розумніша» за інструктивну?

Ні. Кількість токенів лише відображає особливість її роботи — «звичку» спершу вести внутрішній діалог. Розумнішою модель робить якість цих роздумів на складних задачах, а на простих запитаннях велика кількість токенів тільки сповільнює відповідь.

5. Як пояснити різницю у швидкості генерації (токенів за секунду) між моделями qwen3:4b та llama3.2:3b?

Причина в розмірі моделей, тобто в кількості ваг. У qwen3:4b близько 4 млрд параметрів [7], а в llama3.2:3b — 3 млрд [8], тож на кожен токен Qwen процесору доводиться виконувати

більше обчислень, і швидкість падає (приблизно 21 токен/с проти 34 токенів/с за даними Таблиці 2).

Висновок за завданням 1: Міркувальні моделі добре підходять для задач, які потребують багатоетапного логічного виведення: прихований блок thinking допомагає уникнути інтуїтивних помилок (як у задачі з ручкою). Для простих інтерактивних чат-ботів і перевірки фактів доцільніше брати інструктивні моделі — вони відповідають швидше й не витрачають ресурси на очевидні речі.

Завдання 2. Роль system-повідомлення: керування поведінкою моделі

Таблиця 3: Вплив system-повідомлення

Модель	**System**	**Довжина відповіді, символів**	**Мова відповіді**	**Кількість речень**	**Інструкцію виконано?**
llama3.2:3b	лаконічний асистент	1877	укр. (з русизмами)	12	Ні (занадто довга)
llama3.2:3b	українською, 2 речення	316	укр. (з англізмами)	2	Так
qwen3:4b	лаконічний асистент	124	чиста українська	2	Так
qwen3:4b	українською, 2 речення	139	чиста українська	1	Так

Таблиця 4: Витрати токенів

Модель	**System**	**prompt_tokens**	**completion_tokens**	**total_tokens**
llama3.2:3b	лаконічний асистент	50	588	638
llama3.2:3b	українською, 2 речення	54	82	136
qwen3:4b	лаконічний асистент	64	1654	1718
qwen3:4b	українською, 2 речення	64	691	755

Примітка. У llama3.2:3b значення prompt_tokens зросло (50 → 54), у qwen3:4b не змінилося (64 → 64); пояснення наведено у відповіді на питання 3.

Відповіді на питання

1. Яка модель точніше дотрималась обмеження «максимум двома реченнями»?

Найточніше обмеження виконала qwen3:4b: вона видала чітку відповідь з одного-двох речень без зайвих пояснень. Модель llama3.2:3b спершу проігнорувала обмеження й вклалася в ліміт лише після суворішої вказівки.

2. Чи були випадки, коли модель відповіла не українською, попри пряму вказівку?

Так. Локальна інструктивна модель llama3.2:3b нерідко вживала русизми та англійські слова та перекручені форми («relation», «Metodem»), що пов’язано з обмеженістю її словникового запасу.

3. Як змінився prompt_tokens при зміні system-повідомлення і чому?

Значення prompt_tokens залежить від токенізатора моделі та від службових токенів шаблону чату [2]. У llama3.2:3b воно зросло з 50 до 54: суворіше system-повідомлення («українською, 2 речення») розбивається на більшу кількість токенів, ніж «лаконічний асистент». У qwen3:4b значення не змінилося (64 і 64). Імовірно, обидві короткі фрази дали в словнику Qwen однакову кількість токенів, а решту значення становлять службові токени шаблону

чату, однакові в обох запусках; окремо ця гіпотеза не перевірялася. Що кількість токенів залежить від токенізатора, видно і із Завдання 3: для того самого запитання llama3.2:3b показала 43 prompt_tokens, а gpt-oss-20b — 89.

4. Чому в qwen3:4b значення completion_tokens значно більше, ніж у llama3.2:3b, навіть при однаковій довжині видимої відповіді?

qwen3:4b — міркувальна модель, тому сотні токенів вона витрачає на прихований блок роздумів (thinking) і лише після нього формує коротку видиму відповідь [7].

Відповіді чат-бота

1. Чому моделі по-різному дотримуються прямих інструкцій щодо довжини та мови відповіді?

Це залежить від архітектури моделі та обсягу й складу навчальних даних. Міркувальна модель спочатку аналізує інструкцію у прихованому блоці, що допомагає точніше виконати обмеження, а якість мови напряму залежить від частки україномовних текстів у навчальній вибірці.

2. Чим system-повідомлення відрізняється від user-повідомлення з точки зору моделі — чи справді воно має більший пріоритет?

System-повідомлення виконує роль «базових налаштувань»: воно задає тон, стиль і обмеження для всього діалогу. Під час навчання модель привчають сприймати його як правило від розробника, тому зазвичай воно має вищий пріоритет за поточні запити користувача.

3. Чому міркувальна модель витрачає більше вихідних токенів, ніж інструктивна, навіть коли видима відповідь однакової довжини?

Міркувальна модель завжди будує прихований логічний ланцюжок (Chain of Thought) [4] перед фінальним текстом. Ці «невидимі» токени обчислюються так само, як і звичайні, тому показник completion_tokens різко зростає.

4. Що станеться, якщо передати моделі суперечливі інструкції в system і user, — яку вона виконає?

Здебільшого сучасні моделі віддадуть перевагу інструкції з user-повідомлення, адже воно стоїть ближче до кінця контексту й сприймається як найсвіжіше. Втім, результат непередбачуваний: модель може заплутатися, спробувати поєднати обидві вказівки або порушити формат відповіді.

5. Чому малі мовні моделі часто гірше дотримуються інструкцій щодо мови відповіді, ніж великі?

Малі моделі мають обмежену ємність (кількість параметрів), а навчають їх переважно на англомовних даних. Через брак українських текстів у вагах вони гірше засвоюють граматику й легко переходять на суржик чи англійську, коли не знають точного відповідника.

Висновок до завдання 2: System-повідомлення — зручний базовий інструмент керування поведінкою моделі, проте воно не гарантує стовідсоткової надійності. Експеримент показав, що малі моделі часто ігнорують пряму текстову інструкцію, особливо щодо суворих обмежень формату й обсягу. Самого промпту недостатньо для стабільної роботи системи: якщо модель порушує правила, варто перейти на міркувальну модель, додати приклади (few-shot prompting) або програмно обмежити формат виводу.

Завдання 3. Локальна vs хмарна модель: єдиний інтерфейс

Таблиця 5: Порівняння локальної та хмарної моделі

Показник	**Локальна (llama3.2:3b)**	**Хмарна (openai/gpt-oss-20b)**
Час відповіді, с	10.43 с	0.64 с
completion_tokens	56	259 (з них 136 — роздуми)
Довжина відповіді, символів	201	330
Мова відповіді	укр. (з суржиком)	ідеальна українська
Фактичних помилок	Так (помилка контексту)	Немає

Довжину відповіді виміряно функцією len() (для хмарної моделі враховано markdown-розмітку). Моделі: llama3.2:3b [8], openai/gpt-oss-20b [11].

Відповіді на питання

1. Скільки рядків коду довелося змінити, щоб перейти з локальної моделі на хмарну?

Змінювати довелося лише конфігурацію: base_url, api_key і назву model (фактично три рядки у словнику providers.py). Код надсилання запиту (client.chat.completions.create) залишився без змін завдяки сумісності API [1, 3].

2. Яка відповідь повніша й точніша за змістом? Наведіть конкретний приклад різниці.

Хмарна модель відповіла помітно повніше й послідовніше. Локальна модель сплутала контекст і видала загальні фрази про «relation між змінними» та «чергування методів», тоді як хмарна чітко розкрила різницю між функцією та методом у програмуванні (метод — це функція, прив’язана до класу чи об’єкта, з доступом до його стану).

3. Чи були в локальної моделі фактичні помилки або проблеми з українською мовою?

Так, із мовою були серйозні проблеми: суржик, русизми та змішування мов («relation», «Методем», «цели»), а також періодична втрата зв’язку між реченнями. Хмарна модель видала цілком грамотний український текст.

4. Хмарна модель відповіла швидше за локальну — за рахунок чого, якщо запит летить через інтернет?

Хмарна модель працює на потужних серверних кластерах зі спеціалізованими прискорювачами (наприклад, LPU від Groq чи H100). Їхня обчислювальна потужність із запасом перекриває затримку передачі даних мережею.

Відповіді чат-бота

1. Чому формат запитів OpenAI став стандартом, який підтримують інші провайдери, зокрема локальний сервер Ollama?

OpenAI першою запропонувала зручний масовий API [1], навколо якого швидко виросла екосистема бібліотек та інструментів. Іншим провайдерам, зокрема Ollama [3], було вигідно підтримати цей формат, щоб розробники могли переходити до них без переписування коду.

2. Наскільки розмір моделі (3 млрд проти хмарної 20–70 млрд) впливає на якість відповіді порівняно з іншими факторами?

Розмір моделі — один із найважливіших чинників. Якість навчальних даних теж має значення, але мала модель просто не має достатньої ємності, щоб зберегти тонкощі граматики неанглійських мов (зокрема української) і втримувати складні логічні ланцюжки.

3. Хмарна модель відповіла швидше за локальну, попри затримку мережі. Чим це пояснюється?

Хмарні провайдери використовують обладнання, спеціально розроблене для інференсу нейромереж. Домашній комп'ютер, навіть із потужною ігровою відеокартою, значно поступається йому за пропускною здатністю пам'яті.

4. Які ризики для приватності виникають при використанні хмарних API і що з цим роблять у комерційних проєктах?

Основний ризик — передача конфіденційних даних клієнтів або закритого коду на сторонні сервери. У комерційних проєктах його зменшують анонімізацією даних перед надсиланням, жорсткими договорами з провайдерами (NDA/BAA) або використанням локальних моделей у власному контурі.

5. Що таке rate limit у безкоштовних тарифах і як розробники обходять це обмеження?

Rate limit — це обмеження від провайдера на кількість запитів або токенів за хвилину (ми зіткнулися з помилкою 429 на Groq) [10]. Розробники справляються з ним через паузи й повторні спроби в коді (exponential backoff), чергування кількох безкоштовних ключів або перехід на платний тариф.

Висновок до завдання 3: Локальну модель, попри її слабкість, я обирав би лише тоді, коли потрібна повна приватність (аналіз медичних чи фінансових даних, закритий корпоративний код) або відсутній доступ до інтернету. В усіх інших випадках перевага за хмарною моделлю: вона дає швидку генерацію, бездоганну українську, здатність до складного аналізу й стабільну роботу зі структурованими форматами.

Завдання 4. Вимірювання швидкості: TTFT і залежність від довжини запиту

Таблиця 7: Результати вимірювання швидкості

Розміщення	**Модель**	**Промпт**	**TTFT, с**	**Всього, с**	**Симв./с**
локальна	llama3.2:3b	короткий	2.077	2.466	16.4
локальна	llama3.2:3b	середній	2.081	3.777	68.0
локальна	llama3.2:3b	довгий	2.100	16.376	127.7
локальна	qwen3:4b	короткий	15.939	23.204	13.3
локальна	qwen3:4b	середній	84.622	90.495	2.6
локальна	qwen3:4b	довгий	88.380	105.884	13.6
хмарна	openai/gpt-oss-20b	короткий	0.387	0.403	73.7
хмарна	openai/gpt-oss-20b	середній	0.357	0.582	1108.6
хмарна	openai/gpt-oss-20b	довгий	0.393	1.391	1908.3

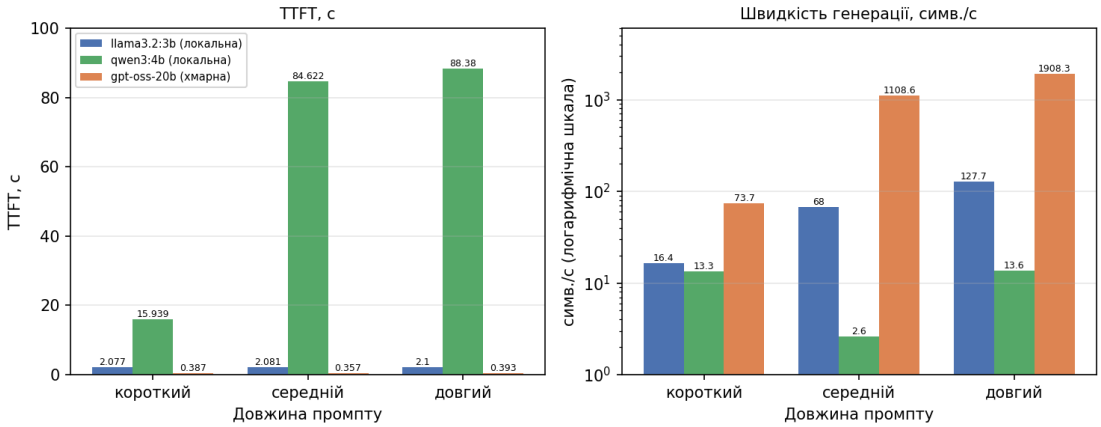


Рисунок 1 — TTFT та швидкість генерації (симв./с) залежно від довжини промпту (за даними Таблиці 7)

На рисунку 1 видно, що TTFT майже не залежить від довжини промпту в llama3.2:3b (≈2.1 с) і gpt-oss-20b (≈0.4 с), тоді як швидкість у симв./с зростає з довжиною відповіді: фіксований час очікування першого токена розподіляється на більший обсяг тексту. У qwen3:4b TTFT становить 16–88 с через приховане міркування перед першим видимим символом. Для швидкості застосовано логарифмічну шкалу, бо значення відрізняються на два порядки.

Таблиця 8: Холодний і теплий старт локальної моделі

Показник	**Холодний старт**	**Теплий старт**
TTFT, с	8.52	2.08
Всього, с	8.81	2.37

Відповіді на питання

1. У якої моделі кращий TTFT і чому саме в неї?

Кращий TTFT показала хмарна модель (0.36–0.39 с проти приблизно 2.1 с у llama3.2:3b). Вона працює на спеціалізованому серверному обладнанні, яке обробляє вхідний контекст майже миттєво й із запасом компенсує мережеву затримку. Локальна qwen3:4b має TTFT 16–88 с, бо до першого видимого символу вона виконує приховане міркування (thinking) [4], тому її TTFT визначається не лише обробкою промпту.

2. Чи збігається «найшвидша за TTFT» з «найшвидшою за символами/с»? Якщо ні — поясніть.

Так, в нашому експерименті ці показники збіглися: хмарна модель виграла і за часом до першого символу, і за загальною швидкістю генерації.

3. Як змінилося співвідношення локальної та хмарної моделі при переході від короткого промпту до довгого?

Розрив у швидкості генерації (симв./с) істотно збільшився на користь хмарної моделі: на короткому промпті вона була швидшою приблизно в 4,5 раза (73.7 проти 16.4 симв./с для llama3.2:3b), а на довгому — уже приблизно в 15 разів (1908.3 проти 127.7 симв./с).

4. Наскільки різняться три повтори одного виміру? Чи достатньо було б одного запуску?

Значення помітно коливалися через зміну навантаження на локальний процесор і стрибки мережевої затримки. Одного запуску недостатньо, бо він може потрапити на випадковий сплеск навантаження, тому ми брали медіану.

5. Чому холодний старт повільніший за теплий?

Під час холодного старту файл ваг моделі (кілька гігабайтів) завантажується з диска в оперативну пам’ять, що потребує часу. Під час теплого старту модель уже в пам’яті й готова до обчислень.

Відповіді чат-бота

1. Що фізично відбувається під час TTFT — часу до першого токена — і чому цей показник вимірюють окремо від загальної швидкості генерації?

Протягом TTFT модель паралельно обробляє весь вхідний промпт (обчислення attention, формування кешу), ще не генеруючи нових слів. Показник вимірюють окремо, бо саме він визначає відчуття «чуйності» системи для користувача, тоді як загальна швидкість характеризує ефективність безперервної видачі тексту.

2. Чому хмарна модель може мати гірший TTFT, але кращу швидкість генерації, ніж локальна?

Гірший TTFT у хмарної моделі можливий через мережеві затримки: пінг до віддаленого сервера може становити кілька сотень мілісекунд. Проте після цього серверні GPU генерують кожен наступний токен у рази швидше за відеокарту локального комп’ютера.

3. Чому перший запуск локальної моделі повільніший за наступні і що таке холодний старт?

Холодний старт — це ініціалізація, під час якої система виділяє відеопам’ять і копіює мільярди параметрів моделі зі сховища (SSD) у VRAM. У наступних запусках ваги вже знаходяться у швидкій пам’яті, тому читання з диска не потрібне.

4. Чому при вимірюванні продуктивності беруть медіану кількох запусків, а не одне значення?

Комп’ютерна система працює не в ідеальних умовах: фонові процеси ОС, коливання затримки мережі чи тимчасове навантаження на сервери можуть спотворити окремий вимір. Медіана відсікає випадкові викиди й показує типовий результат.

5. Чому обробка вхідного запиту йде швидше, ніж генерація відповіді такої ж довжини?

Промпт обробляється паралельно: завдяки архітектурі Transformer [5] матричні множення для всіх його токенів виконуються одночасно. Генерація ж авторегресійна, тобто послідовна: щоб отримати наступне слово, модель має завершити обчислення для всіх попередніх.

Висновок до завдання 4: Для інтерактивного чат-бота критичним є TTFT (Time To First Token): користувач погано сприймає очікування понад 1–2 секунди, а швидкий старт створює відчуття «живого» спілкування. Для фонової обробки великих обсягів тексту (підсумовування книг, аналіз логів) TTFT майже не має значення, і головним стає показник символів за секунду (або токенів/с), що визначає пропускну здатність системи та час обробки великих масивів даних.

Завдання 5. Оцінювання якості моделей за рубрикою

Таблиця 13: Оцінка якості за рубрикою

Завдання	**llama3.2:3b**	**Хмарна (openai/gpt-oss-20b)**	**qwen3:4b**
1. Факт	2	2	2
2. Формат	2	2	2

3. Стиснення	0	2	2
4. Код	1	1	2
5. Логіка	0	2	2
6. Українська	0	2	2
Разом /12	5 / 12	11 / 12	12 / 12

Таблиця 14: Розбіжність оцінок між двома прогонами (так / ні)

Завдання	**llama3.2:3b**	**Хмарна (openai/gpt-oss-20b)**	**qwen3:4b**
1. Факт	ні	ні	ні
2. Формат	так	ні	ні
3. Стиснення	так	так	так
4. Код	ні	так	ні
5. Логіка	так	так	так
6. Українська	так	так	так

Відповіді на питання

1. Яка модель набрала найбільше балів і на яких саме завданнях виникла основна різниця?

Найбільше балів набрала локальна міркувальна модель qwen3:4b (12/12), далі хмарна openai/gpt-oss-20b (11/12), а інструктивна llama3.2:3b значно відстала (5/12). Основна різниця виникла в логіці (завдання 5), збереженні чисел при стисненні (завдання 3) і якості українського тексту (завдання 6), з якими llama3.2:3b не впоралась.

2. На яких завданнях оцінки двох прогонів розійшлися? Що це говорить про надійність моделі?

Найбільше оцінки розійшлися у завданнях зі стисненням, логікою та українською мовою (а в хмарної моделі ще й у коді). Це свідчить про недетермінованість моделей: їхня надійність не абсолютна, і результат (особливо оформлення коду чи вибір мови) може змінюватися при повторному запуску.

3. Чи виконала хоч одна модель вимогу «без markdown» у завданні 2?

Так, усі три моделі виконали цю вимогу в завданні 2 і видали чистий JSON. Водночас звичка до markdown виявилася сильною: у завданні 4 (код) хмарна модель в обох прогонах загорнула код у markdown-блок, проігнорувавши інструкцію «лише код». Чистий код без обгортки видали llama3.2:3b і qwen3:4b, але функція llama3.2:3b була неправильною (швидкість світла задана як 300 «км/ч»), тому вона отримала лише 1 бал.

4. Чи були проблеми з українською мовою — русизми, кальки, перехід на іншу мову?

Хмарна модель і локальна qwen3:4b впоралися без зауважень. Натомість llama3.2:3b мала серйозні проблеми: суржик і русизми («Чтобы найти часовий інтервал полета»), англійські слова («relation»), перекручені слова з латинськими вкрапленнями («зONDом», «Metodem», «Визначátka») та безглузді пояснення (наприклад, «водяна планета» і «пісочна планета» замість різниці між зорею та планетою).

Приклад провалу

Найяскравіший приклад невдачі локальної моделі (llama3.2:3b) із мого JSON-файлу:

Запит: «Зонд летить зі швидкістю 60000 км/год, відстань 480 млн км. Скільки повних діб триватиме політ? Покажи хід розв’язання.»

Відповідь моделі (перший прогін, уривок): «60000 км / 1 доба = 60000 км/добу … 480000000 км / 60000 км/добу = 8000 доб. Окончальний результат: політ триватиме 8000 діб.» У другому прогоні модель взагалі зациклилась: видала близько 400 пунктів із повторюваними фразами на кшталт «Визначайте часовий інтервал полета зондом» та «Роздільте часовий інтервал на повні дні».

Правильна відповідь: 333 повні доби (480 000 000 / 60 000 = 8000 годин; 8000 / 24 ≈ 333,3). Модель отримала 8000 «діб», тобто просто підставила годинну швидкість як добову й не переводила години в доби.

Чому так сталося: У локальної моделі немає вбудованого калькулятора чи математичного модуля: вона лише генерує слова, які статистично часто стоять поруч у навчальних текстах із математики. Вона не розрізнила години й доби вже на першому кроці, а в другому прогоні потрапила в безглуздий цикл повторів, імітуючи «хід розв’язання» без жодного логічного змісту.

Відповіді чат-бота

1. Які ще підходи до оцінювання якості мовних моделей використовують на практиці?

На практиці застосовують автоматизовані бенчмарки (наприклад, MMLU для перевірки знань і HumanEval для коду), підхід «LLM-as-a-Judge», коли сильніша модель оцінює відповіді слабших, а також сліпе A/B-тестування на реальних користувачах (наприклад, Chatbot Arena) [9].

2. Чому мовні моделі часто ігнорують пряму інструкцію не використовувати markdown при генерації JSON?

У даних, на яких навчають моделі (StackOverflow, GitHub), майже весь код і JSON оформлені у markdown-блоки. Ця статистична звичка закріплена у вагах настільки міцно, що може переважити пряму текстову вказівку користувача.

3. Чому малі мовні моделі гірше пишуть українською, ніж англійською?

Навчальні вибірки малих моделей здебільшого англомовні. Через обмежену кількість параметрів багатомовність страждає: модель не здатна вмістити і глибоке розуміння граматики, і багатий словник української мови.

4. Чому та сама модель на однаковий запит може дати відповіді різної якості при повторному запуску?

Сучасні мовні моделі генерують текст імовірнісно. Якщо temperature > 0, під час повторного запуску модель може обрати інший синонім на початку речення, і все подальше формулювання та хід думки підуть іншим шляхом.

5. Що таке галюцинація мовної моделі і чому модель радше вигадає факт, ніж визнає незнання?

Галюцинація — це правдоподібне, але хибне або вигадане твердження моделі. Вона завжди намагається продовжити текст найімовірнішими словами, тож якщо потрібних знань у вагах немає, вона конструює «схожу на правду» відповідь замість того, щоб визнати незнання.

Висновок до завдання 5: Оцінювання за рубрикою виявилося надійним методом, адже замінило суб’єктивне враження («ця модель здалася розумнішою») на конкретні числа (11/12 проти 5/12). Найпоказовішими були завдання з логіки та генерації українського тексту, де локальна llama3.2:3b зазнала повної невдачі. Тому для серйозної роботи з

україномовним текстом я обрав би хмарну модель: вона забезпечує граматичну правильність, добре розуміння логіки та відсутність несподіваних переходів на інші мови чи суржик.

Завдання 6. Керування випадковістю: temperature і top_p

Таблиця 16: Вплив параметрів на різноманітність відповідей

Параметр	**Значення**	**Унікальних із 5 (творче)**	**Правильних із 5 (фактичне)**
temperature	0.0	2	5
temperature	0.7	5	—
temperature	1.5	5	1
top_p	0.1	1	—
top_p	0.5	5	—
top_p	1.0	5	—

Таблиця 17: Перевірка відтворюваності

Показник	**Значення**
Кількість запусків	10
temperature	0.0
Унікальних відповідей	1
Усі відповіді однакові (так / ні)	так

Відповіді на питання

1. Яке значення температури ви поставите агенту, який має повертати JSON, і чому?

Я встановлю temperature = 0.0. Це забезпечує максимальну детермінованість і мінімізує ризик того, що модель випадково додасть зайві символи чи пояснення, які зламають структуру JSON.

2. Яке — генератору варіантів заголовків?

Для генератора заголовків підійде temperature = 0.7: вона дає креативні та різноманітні варіанти, зберігаючи їхню логічність і доречність.

3. Чим вплив top_p відрізняється від впливу temperature за характером, а не за силою?

Temperature перерозподіляє ймовірності між усіма токенами (робить розподіл «гострішим» або рівнішим), а top_p відсікає «хвіст» найменш імовірних токенів, повністю забороняючи їх вибір [6].

4. Чи означає temperature=0 повну відтворюваність результату? Що показав ваш експеримент із 10 запусків?

В експерименті всі 10 відповідей збіглися, тобто відтворюваність була повною. Проте в загальному випадку абсолютної гарантії немає через особливості операцій з плаваючою комою на відеокартах.

5. Наведіть найбезглуздішу відповідь, яку модель видала при temperature=1.5, і поясніть, чому вона взагалі могла з'явитися.

Найабсурднішою була відповідь, що перший iPhone випустила «Google у Лос-Анджелесі», з вкрапленнями арабських слів («с 2007 ةنس»). Висока температура згладила розподіл імовірностей, тож модель могла обрати малоймовірні, випадкові слова замість очевидних.

Відповіді чат-бота

1. Що саме математично робить параметр temperature із розподілом імовірностей наступного токена?

Temperature ділить сирі оцінки моделі (логіти) перед застосуванням функції softmax [1]. Якщо temperature < 1, розподіл стає «гострішим» і модель упевненіше обирає лідерів; якщо temperature > 1, розподіл вирівнюється, і ймовірності токенів стають більш схожими.

2. Чим top_p відрізняється від temperature — обидва впливають на різноманітність, але за яким механізмом?

Top_p (nucleus sampling) [6] сортує токени за ймовірністю й залишає лише ті, що входять до заданої сумарної ймовірності (наприклад, топ-90%). Отже, temperature змінює самі значення ймовірностей, а top_p працює як жорсткий фільтр, що обмежує вибір.

3. Чи є сенс змінювати обидва параметри одночасно, чи краще налаштовувати щось одне?

Документація більшості провайдерів (зокрема OpenAI) радить змінювати лише один із двох параметрів [1]. Обидва впливають на один і той самий процес вибору наступного токена, тому одночасна зміна робить поведінку моделі важко передбачуваною.

4. Чому навіть при temperature=0 мовна модель не гарантує абсолютно однакову відповідь при повторних запусках?

Навіть при temperature = 0 (greedy decoding) графічні процесори оптимізують паралельні матричні обчислення, і операції з плаваючою комою можуть давати мікроскопічні розбіжності між запусками. Іноді це призводить до вибору іншого токена, коли ймовірності майже рівні.

5. Які значення temperature використовують на практиці для витягу структурованих даних, а які для генерації творчого тексту?

Для витягу структурованих даних (JSON, XML, SQL) і написання коду зазвичай беруть temperature = 0.0, щоб уникнути помилок у синтаксисі. Для копірайтингу, перекладів і брейншторму оптимальними вважають значення 0.7–0.9.

Висновок до завдання 6: У наступних лабораторних роботах, де агенти повинні повертати строго структуровані дані (наприклад, виклики інструментів у форматі JSON), я використовуватиму temperature = 0.0. Значення за замовчуванням (зазвичай близько 0.7) розраховане на креативність і живе спілкування. Якщо залишити його для агентських завдань, модель рано чи пізно згенерує зайвий символ або змінить ключі формату, що призведе до помилки парсера та збою програми.

ЗАГАЛЬНИЙ ВИСНОВОК

Таблиця 18: Підсумкове порівняння моделей

Критерій	**Локальна (3B) (llama3.2:3b)**	**Хмарна**	**Локальна більша (4B/8B) (qwen3:4b)**
Швидкість генерації	Повільна (16–128 симв./с)	Надшвидка (74–1908 симв./с)	Дуже повільна (2.6–13.6 симв./с)

TTFT	Високий (~2–9 с)	Низький (~0.4 с)	Дуже високий (~15–88 с)
Якість, балів /12	5 / 12	11 / 12	12 / 12
Стабільність формату	Добра (видає сирий JSON)	Середня (додає markdown)	Відмінна (видає сирий JSON)
Потрібні ресурси	Високі (8+ ГБ RAM/VRAM)	Мінімальні (лише браузер/інтернет)	Дуже високі (16+ ГБ RAM)
Вартість	Безкоштовно	Безкоштовний ліміт / Платно	Безкоштовно
Куди йдуть ваші дані	Залишаються на ПК	Відправляються на сервери провайдера	Залишаються на ПК
Працює без інтернету	Так	Ні	Так
Коли обирати	Локальну модель слід обирати лише тоді, коли критично важлива повна конфіденційність і безпека даних (медичні картки, закритий корпоративний код, банківська таємниця) або система має працювати автономно. Ми свідомо погоджуємося на повільну швидкість, суржик і слабку логіку заради повного контролю над даними.	Хмарну модель слід обирати для решти production-задач, де потрібні висока якість тексту (бездоганна українська), розв’язання складних логічних і математичних задач, стабільна генерація коду та JSON і швидка відповідь (низький TTFT). Ціна цього — залежність від інтернету та лімітів провайдера й передача даних назовні.	Локальну міркувальну модель слід обирати для складних математичних і логічних задач, що потребують повної приватності або автономної роботи. Ми свідомо жертвуємо швидкістю (дуже довгий TTFT) заради якості, граматики та логіки, не відправляючи дані в інтернет.

ВИСНОВОК

Під час виконання лабораторної роботи було розгорнуто локальне та хмарне середовища для роботи з великими мовними моделями (LLM) і проведено порівняльний аналіз моделей різної архітектури (міркувальних та інструктивних) і різного масштабу (локальні на 3–4 млрд параметрів та хмарні на 20+ млрд параметрів). За отриманими експериментальними даними можна зробити такі висновки:

1. Міркувальні vs інструктивні моделі. Міркувальні моделі (на прикладі qwen3:4b) генерують прихований ланцюжок думок (Chain of Thought) [4], через що вони значно повільніші й «дорожчі» за кількістю вихідних токенів, але мають явну перевагу в логічних і математичних задачах. Для простих фактичних запитань їх використання недоцільне.

2. Керування поведінкою (system prompt). System-повідомлення задає базовий формат, але малі локальні моделі часто ігнорують прямі обмеження (обсяг, формат, заборона markdown). Для машинної зчитуваності відповідей (наприклад, JSON) самої текстової інструкції недостатньо.

3. Швидкість і TTFT. Хмарні моделі, попри затримки мережі, значно випереджають локальні як за часом до першого токена, так і за загальною пропускною здатністю (симв./с). Причина — потужні промислові прискорювачі.

4. Оцінка якості та галюцинації. Оцінювання за фіксованою рубрикою підтвердило перевагу хмарної моделі. Локальна llama3.2:3b продемонструвала схильність до галюцинацій у складних розрахунках і серйозну нестачу знань української граматики (суржик, слова з інших мов).

5. Параметри генерації. Temperature — критичний параметр керування [1]. Для інтеграції моделей у програмний код (отримання JSON) слід використовувати temperature = 0.0 заради детермінованості, а для творчих завдань оптимальні значення 0.7–1.0.

Головний підсумок: у сучасній інженерії не існує універсально «найкращої» моделі, вибір завжди є компромісом. Локальні моделі незамінні для роботи в ізольованих середовищах і збереження приватності корпоративних даних, тоді як хмарні API є стандартом для побудови швидких, надійних та інтелектуально складних production-систем, орієнтованих на кінцевого користувача.

СПИСОК ЛІТЕРАТУРИ

[1] OpenAI. API Reference: Chat Completions (параметри temperature, top_p) [Електронний ресурс]. URL: https://platform.openai.com/docs/api-reference/chat

[2] Ollama. API documentation (поля prompt_eval_count, eval_count, eval_duration) [Електронний ресурс]. URL: https://github.com/ollama/ollama/blob/main/docs/api.md

[3] Ollama. OpenAI compatibility [Електронний ресурс]. URL: https://github.com/ollama/ollama/blob/main/docs/openai.md

[4] Wei J. et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models // Advances in Neural Information Processing Systems (NeurIPS), 2022. arXiv:2201.11903.

[5] Vaswani A. et al. Attention Is All You Need // Advances in Neural Information Processing Systems (NeurIPS), 2017. arXiv:1706.03762.

[6] Holtzman A. et al. The Curious Case of Neural Text Degeneration // International Conference on Learning Representations (ICLR), 2020. arXiv:1904.09751.

[7] Yang A. et al. Qwen3 Technical Report, 2025. arXiv:2505.09388.

[8] Meta AI. Llama 3.2 Model Card [Електронний ресурс]. URL: https://www.llama.com/docs/model-cards-and-prompt-formats/llama3_2/

[9] Zheng L. et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena // NeurIPS, 2023. arXiv:2306.05685.

[10] Groq. Rate Limits [Електронний ресурс]. URL: https://console.groq.com/docs/rate-limits

[11] OpenAI. gpt-oss-120b & gpt-oss-20b Model Card, 2025. arXiv:2508.10925.